

操作系统第一次作业

姓名: 陈天乐

学号:

Q1:系列计算机思想是1960年代由IBMSystem/360大型主机引入的。这一思想现在还有没有意义?

系列计算机的思想核心是通过统一的架构和标准,使同一系列中的不同计算机型号能够兼容相同的软件和硬件,同时提供不同的性能和功能选项。比如:同一系列中的计算机采用相同的指令集架构,为某一型号开发的程序可以在系列中的其他型号上运行,同时用户还可以根据自己的需要选择不同性能的型号。

这个思想在当今依然具有重要意义。因为它提出了兼容性和标准化,这大大减少了软件工程师面对不同硬件时的移植和开发成本,同时也具有非常强的可拓展性。除了技术方面,在目前计算机市场同样也有着重要的指导作用。

Q2:对程序员而言,系统调用看起来就像是其它库函数调用一样。程序员了解哪个库函数导致系统调用很重要吗?在什么情况下重要?为什么?

在平常设计程序或者完成特定任务时,程序员对于哪个库函数导致系统调用的了解并不重要,因为此时是任务导向型的,程序员关注功能的实现是最重要的。但是当程序员需要优化代码效率、提高代码安全性,或者是编写的代码涉及到从用户态到核心态的切换、程序出现异常时,了解库函数的系统调用是很有必要的。

Q3:系统调用通常会被封装成库函数,那么库函数和实现系统调用的内核函数两者是否必须具有相同的名字?如果不是,哪一个名字对程序员更重要?

不需要,它们的命名通常由不同的上下文和设计目标决定。这两个名字显然库函数的名字对程序员更加重要。首先,程序员直接面向的是库函数,一个更加指向功能的名字更能帮助程序员更好的编写代码;其次,程序员可以通过命令查看库函数中的系统调用,因此库函数的命名更需要被清晰定义。而内核函数的命名则对系统开发者来说更加重要。

Q4:某计算机有一个CPU和一台设备,现有A和B两个程序同时到达,按A、B顺序依次投入运行。A的执行过程:计算20s,使用设备10s,计算10s,使用设备20s,计算10s;B的执行过程:使用设备20s,计算20s,使用设备10s,计算10s,使用设备20s。试计算:

①采用顺序执行时A的周转时间、B的周转时间、CPU的利用率、设备的利用率;

顺序执行说明着A先运行,完成后B再运行。那么A的周转时间 $t_A = 20 + 10 + 10 + 20 + 10 = 70s$, B的周转时间 $t_B = t_A + 20 + 20 + 10 + 10 + 20 = 150s$ 。总时间 $t_{\text{总}} = t_B = 150s$,而CPU处于工作状态的时间 $t_{\text{work}} = 20 + 10 + 10 + 20 + 10 = 70s$,故CPU使用效率 $\alpha_1 = \frac{t_{\text{work}}}{t_{\text{总}}} = 46.67\%$ 。设备使用效率 $\alpha_2 = 1 - \alpha_1 = 53.33\%$ 。

②采用并发执行时A的周转时间、B的周转时间、CPU的利用率、设备的利用率。

顺序执行说明着A、B可以在资源不冲突的前提下同时交错使用CPU和设备。那么A的周转时间

$t_A = 20 + 20$ （需要等待B） $+ 10 + 20 + 10 = 80s$ ，B的周转时间

$t_B = 20 + 20 + 10 + 20$ （需要等待A） $+ 20 = 90s$ 。总时间 $t_{\text{总}} = t_B = 90s$ ，而CPU处于工作状态的时间 $t_{\text{work}} = 20 + 20 + 10 + 10 + 10 = 70s$ ，故CPU使用效率 $\alpha_1 = \frac{t_{\text{work}}}{t_{\text{总}}} = 77.78\%$ 。设备使用时间 $t'_{\text{work}} = 20 + 10 + 10 + 20 + 20 = 80s$ ， $\alpha_2 = 88.89\%$ 。

Q5:UNIX/Linux创建进程分为fork()和exec()两个函数来处理，Windows则只用一个函数CreateProcess()来处理，你对其各自的优缺点有什么看法？

对于UNIX/Linux分开两个函数来操作，优点是更加细分，灵活度更好，并且可以实现代码复用；但是缺点是使得代码的复杂度提高，同时两个函数频繁切换进程很可能会带来很多额外的时间开销。对于Windows的一个函数处理，优点是直观且简洁，降低代码的复杂度，并且有更好的多线程支持；但是缺点也很明显，灵活度较低，并且相较于fork()和exec()两个轻量级函数而言，CreateProcess()的资源开销更大，并且可移植性较低。